

Sistemi informativi su Web

Autenticazione e autorizzazione

aa 2023-2024



Paolo Merialdo
Università degli Studi Roma Tre



This work is licensed under a Creative Commons Attribution-Noncommercial-Share Alike 3.0 United States
See <http://creativecommons.org/licenses/by-nc-sa/3.0/us/> for details

Autenticazione e autorizzazione

- In molte applicazioni è necessario *autenticare* e *autorizzare* l'accesso degli utenti
 - *Autenticazione*: verifica l'identità dell'utente
 - *Autorizzazione*: verifica se l'utente è autorizzato a ottenere la risorsa richiesta
 - ad ogni utente viene associato un ruolo (es. "amministratore", "cliente", etc.)
 - ad ogni risorsa (protetta) viene associato l'insieme dei ruoli autorizzati all'accesso (es. l'operazione di inserimento dei dati può essere fatto solo dagli utenti che hanno il ruolo di "amministratore")

Autenticazione e autorizzazione

- L'autenticazione può essere effettuata tramite username e password
 - Il server registra le credenziali degli utenti noti (username, password, ruolo)
 - L'utente invia al server username e password
 - Se il server conferma l'esistenza di username e password, l'utente è autenticato
- L'autorizzazione si basa sulle credenziali dell'utente autenticato
 - Ad ogni utente è associato un ruolo: il server verifica se il ruolo dell'utente gli permette di accedere alla risorsa richiesta

Autenticazione e autorizzazione

- Concettualmente, possiamo pensare a questa soluzione:
 - Ad ogni utente (registrato) sono associate credenziali di accesso: username, password, ruolo
 - Ogni risorsa protetta è associata ad un ruolo
 - Es.: possono accedere a tutte le risorse con uno specifico path (ad es., /admin/**) solo gli utenti con un ruolo specifico (ad es., ADMIN)
 - Quando il sistema riceve una richiesta, verifica se per la risorsa richiesta sono necessarie autorizzazioni
 - In caso affermativo, il sistema verifica se l'utente si è autenticato
 - In caso negativo (l'utente non si è ancora autenticato)
 - il sistema risponde con una form che raccoglie username e password
 - Attraverso la form il client invia al server username e password
 - Il sistema di autenticazione del server recupera le credenziali dell'utente
 - Se l'autenticazione fallisce, il sistema risponde con una pagina di errore
 - Altrimenti il sistema salva le credenziali in sessione
 - Il sistema verifica se il ruolo dell'utente è compatibile con quello associato alla risorsa
 - In caso affermativo, il sistema evade la richiesta
 - Altrimenti indirizza ad una pagina di errore

Autenticazione e autorizzazione

- Potremmo costruirci una soluzione che implementa tutte le operazioni descritte nella slide precedente
- Ma sarebbe estremamente ripetitivo e, soprattutto, rischioso: un errore comprometterebbe la sicurezza del sistema
- Il framework Spring, con le librerie Spring Security, ci offre un supporto robusto ed affidabile alla gestione di autenticazione e autorizzazione
 - NOTA TECNICA: dobbiamo includere le librerie di Spring Security nel POM (lo possiamo fare direttamente con Spring Initializer quando creiamo il progetto)

Schema soluzione

MainController

GET / semplice inoltro alla pagina

AuthenticController

GET /login presenta la pagina di login

GET /register mostra la form di registrazione

GET /register valida i dati e li salva

GET /default inoltra a due pag diverse in base al ruolo

MovieController

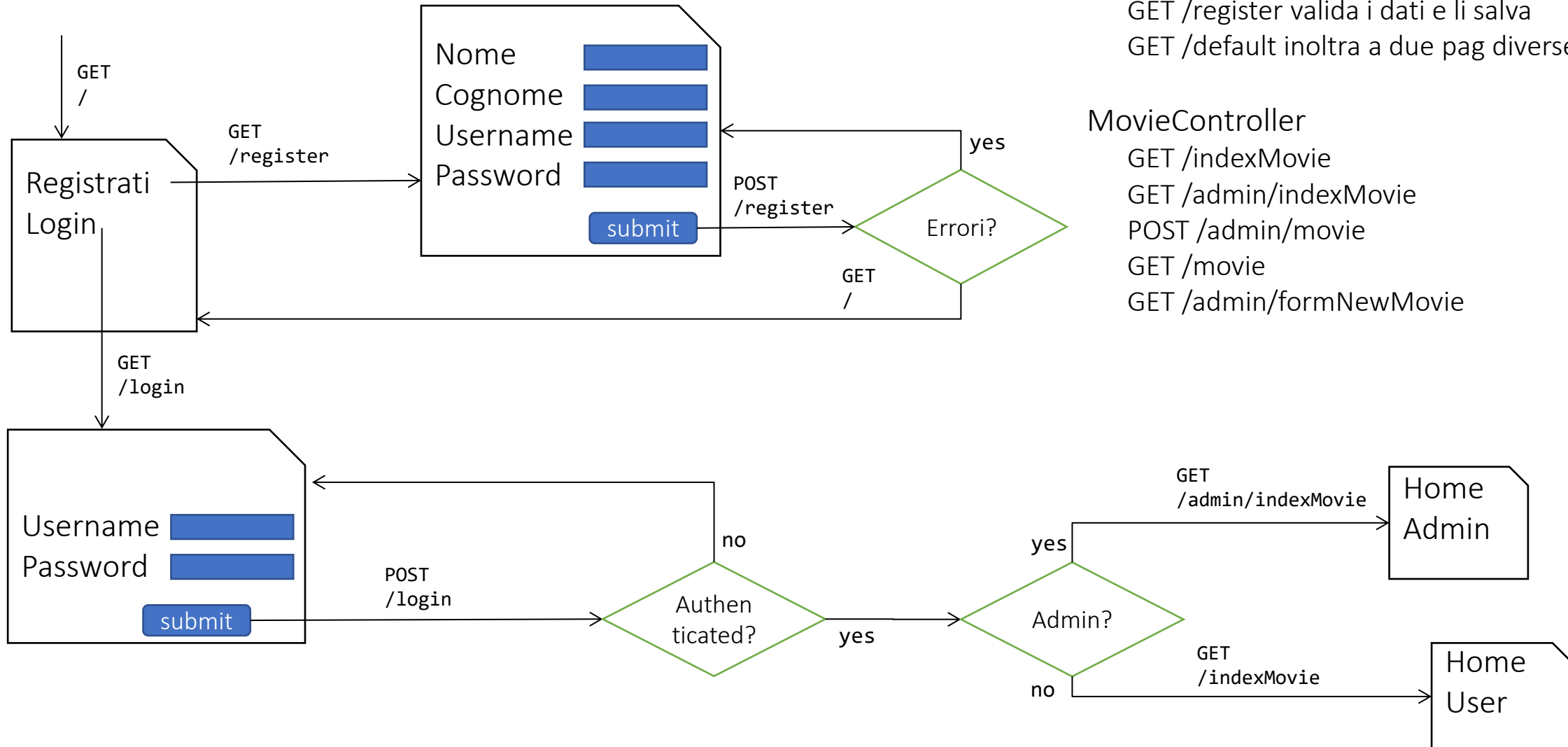
GET /indexMovie

GET /admin/indexMovie

POST /admin/movie

GET /movie

GET /admin/formNewMovie



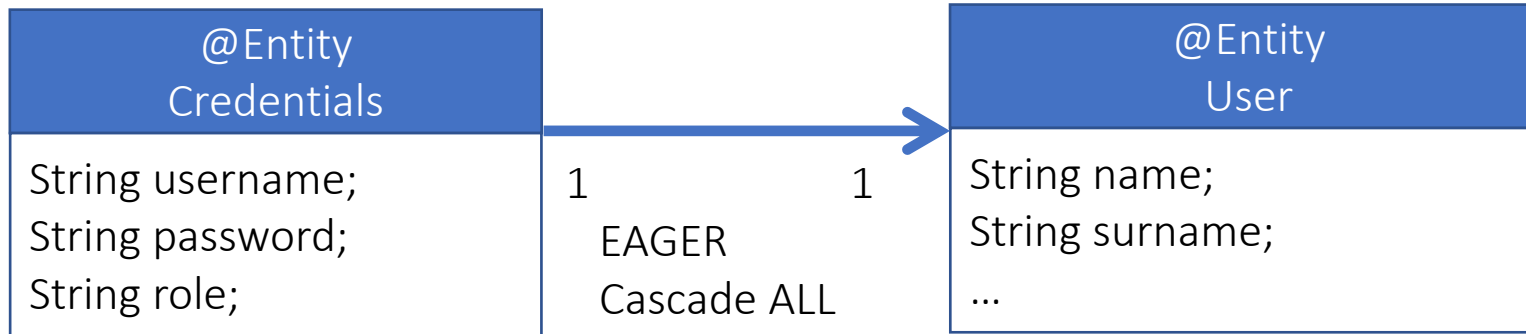
Spring Security

- Attraverso una procedura di configurazione definiamo:
 - Dove vengono memorizzate le credenziali e quali sono le procedure per accedervi
 - Quali sono le risorse protette e a quali ruoli sono associate
 - Quali sono gli url
 - della pagina con la form di login
 - delle pagine di successo/insuccesso del login
 - della pagina di logout
 - delle pagine di successo/insuccesso del logout

Spring Security

- Nelle prossime slide impostiamo una soluzione sufficientemente generica per essere utilizzata (con semplici adattamenti) in diversi contesti
- Introduciamo due classi `@Entity` del modello (`User` e `Credentials`) che ci permettono di adattare la soluzione a contesti reali
- Le slide fanno riferimento al progetto didattico `siw-movie-03`

Classi User e Credentials



- Nella classe **User** avremo tutte le informazioni necessarie a modellare un utente, come definito dai requisiti (ad es. nome, cognome, data e luogo di nascita, indirizzo, telefono, email, etc.)
- Nella classe **Credentials** memorizziamo le credenziali associate all'utente: username, password, ruolo
 - I possibili valori per il ruolo sono predefiniti e dipendono dai requisiti. Ipotizziamo due ruoli: amministratore e utente generico

Classi User e Credentials

- Alle due classi associamo repository e service:
 - UserRepository
 - CredentialsRepository
Al quale aggiungiamo il metodo:
`public Optional<Credentials> findByUsername(String username)`
 - UserService
Con i metodi:
`public User getUser(Long id)`
`public User saveUser(User user)`
 - CredentialsService
Con i metodi:
`public Credentials getCredentials(Long id)`
`public Credentials getCredentials(String username)`
`public Credentials saveCredentials(Credentials credentials)`

Configurazione di Spring Security 5.7

- La configurazione viene implementata tramite una classe che crea un componente (Bean) che filtra le richieste HTTP
 - Nel codice di esempio, vedi la classe **AuthConfiguration**
- Usa DataSource: un componente di Spring che modella un punto di accesso al DB (nel nostro esempio, salviamo le credenziali nel database)
- Usa un oggetto responsabile di criptare il valore della password

Configurazione (Spring Security 5.7)

- Implementiamo i metodi, sovrascrivendo l'implementazione di default di `WebSecurityConfigurerAdapter`:
 - `@Bean`
`protected SecurityFilterChain configure(final HttpSecurity httpSecurity) throws Exception{`
definisce il filtro HTTP che gestisce le policies di autenticazione e autorizzazione attraverso una lunga serie di invocazioni concatenate all'oggetto `HttpSecurity`: i metodi restituiscono lo stesso oggetto `http` (aggiornato dall'invocazione del metodo precedente)
 - `public void configureGlobal(AuthenticationManagerBuilder auth)`
specifica come il sistema deve recuperare username, password e ruoli nel DB
 - `PasswordEncoder passwordEncoder()`
Definisce come salvare in maniera criptata la password nel database

La classe di configurazione (Spring Security 5.7)

@Configuration

@EnableWebSecurity

```
public class AuthConfiguration {
```

@Autowired

```
private DataSource dataSource;
```

@Autowired

```
public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
```

```
    auth.jdbcAuthentication()
```

```
        .dataSource(dataSource)
```

```
        .authoritiesByUsernameQuery("SELECT username, role from credentials WHERE username=?")
```

```
        .usersByUsernameQuery("SELECT username, password, 1 as enabled FROM credentials WHERE username=?");
```

```
}
```

@Bean

```
public PasswordEncoder passwordEncoder(){
```

```
    return new BCryptPasswordEncoder();
```

```
}
```

continua

```

@Bean
protected SecurityFilterChain configure(final HttpSecurity httpSecurity)
    throws Exception{

    httpSecurity
        .csrf().and().cors().disable()
        .authorizeHttpRequests()
        // .requestMatchers("/**").permitAll()
        // chiunque (autenticato o no) può accedere alle pagine index, login, register, ai css e alle immagini
        .requestMatchers(HttpMethod.GET,"/","/index","/register","/css/**", "/images/**", "favicon.ico").permitAll()
        // chiunque (autenticato o no) può mandare richieste POST al punto di accesso per login e register
        .requestMatchers(HttpMethod.POST,"/register", "/login").permitAll()
        .requestMatchers(HttpMethod.GET,"/admin/**").hasAnyAuthority(ADMIN_ROLE)
        .requestMatchers(HttpMethod.POST,"/admin/**").hasAnyAuthority(ADMIN_ROLE)
        // tutti gli utenti autenticati possono accere alle pagine rimanenti
        .anyRequest().authenticated()
        // LOGIN: qui definiamo il login
        .and().formLogin()
        .loginPage("/login")
        .permitAll()
        .defaultSuccessUrl("/success", true)
        .failureUrl("/login?error=true")
        // LOGOUT: qui definiamo il logout
        .and()
        .logout()
        // il logout è attivato con una richiesta GET a "/logout"
        .logoutUrl("/logout")
        // in caso di successo, si viene reindirizzati alla home
        .logoutSuccessUrl("/")
        .invalidateHttpSession(true)
        .deleteCookies("JSESSIONID")
        .logoutRequestMatcher(new AntPathRequestMatcher("/logout"))
        .clearAuthentication(true).permitAll();
    return httpSecurity.build();
}

```

fine

Alcuni dettagli (1)

- Spring security salva le informazioni dell'utente autenticato in un oggetto della classe

`org.springframework.security.core.userdetails.UserDetails`

che possiamo ottenere con il metodo statico

```
SecurityContextHolder.getContext().getAuthentication().getPrincipal();
```

Alcuni dettagli (1b)

- Per fare in modo che lo username sia sempre disponibile ai template, definiamo una classe annotata `@ControllerAdvice` che ci permette di associare un attributo al modello per tutte le richieste

```
@ControllerAdvice
public class GlobalController {
    @ModelAttribute("userDetails")

    public UserDetails getUser() {
        UserDetails user = null;
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        if (!(authentication instanceof AnonymousAuthenticationToken)) {
            user = (UserDetails) SecurityContextHolder.getContext().getAuthentication().getPrincipal();
        }
        return user;
    }
}
```

Alcuni dettagli (2)

- Nella soluzione proposta, gli utenti vengono registrati con il ruolo DEFAULT (vedi metodo `saveCredentials` nella classe `CredentialsService`): sarebbe poco sensato permettere di creare utenze con il ruolo di amministratore
 - Per provare il nostro sistema con diversi utenti, possiamo cambiare il ruolo di un utente (promuovendolo ad amministratore) accedendo direttamente al database (e modificando il valore dell'attributo `role` a `ADMIN`)
 - In un sistema reale dovremmo predisporre una specifica procedura per la registrazione (e la gestione) di utenti con il ruolo di amministratore

Alcuni dettagli (3)

- Se l'utente non è autenticato:
 - errore http “401 Unauthorized”
- Se l'utente non ha il ruolo richiesto:
 - errore http “403 Forbidden”

Alcuni dettagli (4)

- Di default, dopo il login Spring Security indirizza alla risorsa protetta alla quale l'utente ha provato ad accedere. Se vogliamo indirizzarlo sempre ad uno specifico URL, possiamo forzarlo nella configurazione con il metodo `.defaultSuccessUrl()`, impostando il secondo parametro (opzionale) a `true`:

```
.defaultSuccessUrl("/success", true)
```